

Capstone — Compute Economics: The Integrating Summary

A one-page executive brief. One headline number from each of the five models, woven into a single compute-economics narrative — the artifact for an executive or Board discussion.

Answers the two JD bullets the individual models do not: *create a durable cross-functional forum for compute economics and prepare high-quality materials for executive and Board-level discussions.*

Recompute the five headline numbers from the live database

The brief regenerates from the data — nothing is hard-coded.

```
In [1]: import sqlite3, pandas as pd, numpy as np
conn = sqlite3.connect('../data/events.db')

# shared assumptions (see individual models)
HIT_COST_FRACTION = 0.15
PRICE = {'interactive':0.0009, 'batch':0.0004}      # $ / 1K output tokens
TARGET_UTIL, CAPEX = 0.75, 30_000
events = pd.read_sql_query('SELECT COUNT(*) n FROM inference_events', conn).
SCALE = 1_000_000_000 / (events/365)              # sample -> 1B inference
ANNUAL_INFERENCES = 1_000_000_000 * 365

H = {}      # headline numbers

# --- Model 1: ProductC premium to blended cost/1K output tokens -----
q1 = '''SELECT e.product, SUM(e.tokens_output) ot,
            SUM(e.gpu_seconds*c.blended_cost_per_gpu_second) cost
            FROM inference_events e JOIN cost_constants c USING(gpu_type) GROUP
p = pd.read_sql_query(q1, conn)
p['cp1k'] = 1000*p.cost/p.ot
blended = 1000*p.cost.sum()/p.ot.sum()
top = p.loc[p.cp1k.idxmax()]
H['m1'] = (top['product'], 100*(top.cp1k/blended-1))

# --- Model 2: interactive batch tradeoff + SLO-optimal batch -----
q2 = '''SELECT e.batch_size, e.latency_ms, e.gpu_seconds*c.blended_cost_per_
            FROM inference_events e JOIN cost_constants c USING(gpu_type) WHERE
d2 = pd.read_sql_query(q2, conn)
g2 = d2.groupby('batch_size').agg(lat=('latency_ms', 'mean'), cpi=('cpi', 'mea
lo = g2[g2.batch_size.between(2,4)].cpi.mean(); hi = g2[g2.batch_size.betwee
opt = g2[g2.lat<=150].loc[g2[g2.lat<=150].cpi.idxmin(), 'batch_size']
H['m2'] = (100*(1-hi/lo), int(opt))
```

```

# --- Model 3: cache ROI -----
c_miss = pd.read_sql_query(''SELECT e.gpu_seconds*c.blended_cost_per_gpu_se
                        FROM inference_events e JOIN cost_constants c USING(gpu_type)''', co
cur_hr = pd.read_sql_query('SELECT AVG(cache_hit) hr FROM inference_events',
eff = lambda hr: (hr*HIT_COST_FRACTION + (1-hr))*c_miss
pct = 100*(1-eff(cur_hr+0.10)/eff(cur_hr))
sav = (eff(cur_hr)-eff(cur_hr+0.10))*ANNUAL_INFERENCES
H['m3'] = (pct, sav)

# --- Model 4: fleet growth + capex -----
d4 = pd.read_sql_query('SELECT DATE(timestamp) day, SUM(gpu_seconds) gs FROM
d4['t'] = (pd.to_datetime(d4.day)-pd.to_datetime(d4.day).min()).dt.days
slope, intercept = np.polyfit(d4.t, d4.gs, 1)
gpus = lambda daily: daily*SCALE/(86_400*TARGET_UTIL)
g_now = gpus(intercept+slope*d4.t.max())
g_q4 = gpus(intercept+slope*(d4.t.max()+360))
H['m4'] = (g_now, g_q4, (g_q4-g_now)*CAPEX, (intercept+slope*d4.t.max())/(ir

# --- Model 5: margin lift from reallocation -----
q5 = ''SELECT e.workload_type, SUM(e.gpu_seconds) gs, SUM(e.tokens_output)
      SUM(e.gpu_seconds*c.blended_cost_per_gpu_second) cost
      FROM inference_events e JOIN cost_constants c USING(gpu_type) GROUP
m = pd.read_sql_query(q5, conn).set_index('workload_type')
m['rev'] = m.ot/1000*pd.Series(PRICE)
m['rev_per_gs'], m['cost_per_gs'] = m.rev/m.gs, m.cost/m.gs
tot = m.gs.sum(); cur_frac = m.gs['interactive']/tot
def bmargin(f):
    gi, gb = tot*f, tot*(1-f)
    rev = gi*m.rev_per_gs['interactive']+gb*m.rev_per_gs['batch']
    cost = gi*m.cost_per_gs['interactive']+gb*m.cost_per_gs['batch']
    return (rev-cost)/rev
prem = 100*((m.rev-m.cost)['interactive']/m.gs['interactive'])/((m.rev-m.cos
H['m5'] = (prem, bmargin(cur_frac)*100, bmargin(0.90)*100)
H

```

```

Out[1]: {'m1': ('ProductC', np.float64(17.378671398064792)),
        'm2': (np.float64(38.65573641555606), 16),
        'm3': (np.float64(11.848286180251932), np.float64(2252120.165119765)),
        'm4': (np.float64(1260.0378793302646),
               np.float64(1984.9212710865054),
               np.float64(21746501.752687223),
               np.float64(2.3905090781512635)),
        'm5': (np.float64(47.394481886535715),
               np.float64(66.41560552760774),
               np.float64(67.4225690755357))}

```

The five headline insights

```

In [2]: brief = pd.DataFrame([
        ['1. Token Economics',
         f'{H["m1"][0]} runs {H["m1"][1]:.0f}% above blended cost per 1K output tok
        ['2. Inference Economics',
         f'Interactive batch 3->14 cuts cost/inference {H["m2"][0]:.0f}%; margin-op
        ['3. Cache Modeling',
         f'+10 pts cache-hit -> {H["m3"][0]:.1f}% lower effective cost/inference ~

```

```
[ '4. Demand Forecasting',
  f"Demand {H['m4'][3]:.1f}x/yr; fleet {H['m4'][0]:.0f} -> {H['m4'][1]:.0f}
  '5. Capacity Allocation',
  f"Interactive earns {H['m5'][0]:.0f}% more margin/GPU-sec; shift to 90% ir
], columns=['Model', 'Headline insight'])
pd.set_option('display.max_colwidth', None)
brief.style.hide(axis='index')
```

Out[2]:

	Model	Headline insight
	1. Token Economics	ProductC runs 17% above blended cost per 1K output tokens (decode-bound output intensity).
	2. Inference Economics	Interactive batch 3->14 cuts cost/inference 39%; margin-optimal batch under a 150ms SLO is 16.
	3. Cache Modeling	+10 pts cache-hit -> 11.8% lower effective cost/inference ~\$2.3M/yr at 1B inf/day.
	4. Demand Forecasting	Demand 2.4x/yr; fleet 1,260 -> 1,985 GPUs by Q4 (base) = ~\$22M capex.
	5. Capacity Allocation	Interactive earns 47% more margin/GPU-sec; shift to 90% interactive lifts blended margin 66.4% -> 67.4% (+1.0 pt).

The integrated story

cost → cache → demand → allocation → margin

- 1. Cost is set in the decode phase.** Output-heavy products (ProductC, +17% to blended) cost more per token because decode parallelises worst — so cost-per-token is a *throughput* problem, not a volume problem (Model 1).
- 2. Two levers move that cost.** Batching cuts cost ~39% but is capped by the latency SLO (Model 2); **caching cuts cost without a latency penalty** and is therefore the higher-quality lever — each +10 points is ~\$2.25M/yr (Model 3).
- 3. Demand sets the scale of the bet.** Usage compounds ~2.4x/yr, taking the fleet from ~1,260 to ~1,985 GPUs (base) and ~\$22M of take-or-pay capital within a year (Model 4).
- 4. Allocation is nearly optimised, so utilisation is the live lever.** Capacity already favours high-margin interactive (77%), so further mix shifts add only ~1 margin point; the binding constraint is lifting utilisation from ~76% toward the ~85% ceiling without breaking the SLO (Model 5).

The through-line: margin on a compute fleet is won by raising effective throughput per GPU-second — cache first (free), batch to the SLO second, and procure against a forecast band rather than a point.

Decisions this brief would inform

- **Pricing / routing:** reprice or route output-heavy ProductC to higher-throughput GPUs (Model 1).
- **Eng prioritisation:** fund cache-hit improvements — the ~\$2.25M-per-10-points ROI clears most eng costs (Model 3).
- **Capital:** stage GPU procurement against the ~430-GPU downside/upside band rather than a single commit (Model 4).
- **Operations:** target utilisation toward ~85% under the latency SLO as the primary remaining margin lever (Models 2 & 5).

This is the durable, cross-functional view — finance, product, and infra reading the same five numbers — that the compute-economics forum would convene around.